



Equal Subset Sum Partition (medium)

We'll cover the following



- Problem Statement
- Try it yourself
- Basic Solution
 - Code
 - Time and Space complexity
- Top-down Dynamic Programming with Memoization
 - Code
 - Time and Space complexity
- Bottom-up Dynamic Programming
 - Code
 - Time and Space complexity

Problem Statement

Given a set of positive numbers, find if we can partition it into two subsets such that the sum of elements in both subsets is equal.

Example 1:

Input: {1, 2, 3, 4}

Output: True

Explanation: The given set can be partitioned into two subsets with equal sum: {1, 4} & {2, 3}

Example 2:

Input: {1, 1, 3, 4, 7}

Output: True

Explanation: The given set can be partitioned into two subsets with equal sum: {1, 3, 4} & {1, 7}

Example 3:

Input: {2, 3, 4, 6}

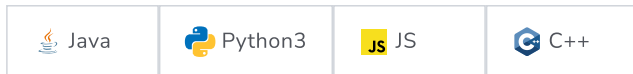
Output: False

Explanation: The given set cannot be partitioned into two subsets with equal sum.

Try it yourself

This problem looks similar to the 0/1 Knapsack problem. Try solving it before moving on to see the solution:





```

1 class PartitionSet {
2
3     static boolean canPartition(int[] num) {
4         //TODO: Write - Your - Code
5         return false;
6     }
7 }

```

Test

Save

Reset



Basic Solution

This problem follows the **0/1 Knapsack pattern**. A basic brute-force solution could be to try all combinations of partitioning the given numbers into two sets to see if any pair of sets has an equal sum.

Assume that **S** represents the total sum of all the given numbers. Then the two equal subsets must have a sum equal to **S/2**. This essentially transforms our problem to: "Find a subset of the given numbers that has a total sum of **S/2**".

So our brute-force algorithm will look like:

```

1 for each number 'i'
2     create a new set which INCLUDES number 'i' if it does not exceed 'S/2', and recursively
3     process the remaining numbers
4     create a new set WITHOUT number 'i', and recursively process the remaining items
5 return true if any of the above sets has a sum equal to 'S/2', otherwise return false

```

Code

Here is the code for the brute-force solution:



```

1 class PartitionSet {
2
3     public boolean canPartition(int[] num) {
4         int sum = 0;
5         for (int i = 0; i < num.length; i++)
6             sum += num[i];
7
8         // if 'sum' is a an odd number, we can't have two subsets with equal sum
9         if(sum % 2 != 0)
10            return false;
11
12        return this.canPartitionRecursive(num, sum/2, 0);
13    }
14
15    private boolean canPartitionRecursive(int[] num, int sum, int currentIndex) {
16        // base check
17        if (sum == 0)
18            return true;
19
20        if(num.length == 0 || currentIndex >= num.length)
21            return false;
22
23        // recursive call after choosing the number at the currentIndex

```

```

24 // if the number at currentIndex exceeds the sum, we shouldn't process this
25 if( num[currentIndex] <= sum ) {
26     if(canPartitionRecursive(num, sum - num[currentIndex], currentIndex + 1))
27         return true;
28 }
29
30 // recursive call after excluding the number at the currentIndex
31 return canPartitionRecursive(num, sum, currentIndex + 1);
32 }
33
34 public static void main(String[] args) {
35     PartitionSet ps = new PartitionSet();
36     int[] num = {1, 2, 3, 4};
37     System.out.println(ps.canPartition(num));
38     num = new int[]{1, 1, 3, 4, 7};
39     System.out.println(ps.canPartition(num));
40     num = new int[]{2, 3, 4, 6};
41     System.out.println(ps.canPartition(num));
42 }
43 }
44

```

Run

Save

Reset



Time and Space complexity

The time complexity of the above algorithm is exponential $O(2^n)$, where 'n' represents the total number. The space complexity is $O(n)$, which will be used to store the recursion stack.

Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems. As stated in previous lessons, memoization is when we store the results of all the previously solved sub-problems so we can return the results from memory if we encounter a problem that has already been solved.

Since we need to store the results for every subset and for every possible sum, therefore we will be using a two-dimensional array to store the results of the solved sub-problems. The first dimension of the array will represent different subsets and the second dimension will represent different 'sums' that we can calculate from each subset. These two dimensions of the array can also be inferred from the two changing values (sum and currentIndex) in our recursive function `canPartitionRecursive()`.

Code

Here is the code for Top-down DP with memoization:

Java

Python3

C++

JS

```

1 class PartitionSet {
2
3     public boolean canPartition(int[] num) {
4         int sum = 0;
5         for (int i = 0; i < num.length; i++)
6             sum += num[i];
7
8         // if 'sum' is a an odd number, we can't have two subsets with equal sum
9         if (sum % 2 != 0)
10             return false;
11

```



Tr



```

12     boolean[][] dp = new boolean[num.length][sum / 2 + 1];
13     return this.canPartitionRecursive(dp, num, sum / 2, 0);
14 }
15
16 private boolean canPartitionRecursive(boolean[][] dp, int[] num, int sum, int currentIndex) {
17     // base check
18     if (sum == 0)
19         return true;
20
21     if (num.length == 0 || currentIndex >= num.length)
22         return false;
23
24     // if we have not already processed a similar problem
25     if (dp[currentIndex][sum] == null) {
26         // recursive call after choosing the number at the currentIndex
27         // if the number at currentIndex exceeds the sum, we shouldn't process this
28         if (num[currentIndex] <= sum) {
29             if (canPartitionRecursive(dp, num, sum - num[currentIndex], currentIndex + 1)) {
30                 dp[currentIndex][sum] = true;
31                 return true;
32             }
33         }
34
35         // recursive call after excluding the number at the currentIndex
36         dp[currentIndex][sum] = canPartitionRecursive(dp, num, sum, currentIndex + 1);
37     }
38
39     return dp[currentIndex][sum];
40 }
41
42 public static void main(String[] args) {
43     PartitionSet ps = new PartitionSet();
44     int[] num = { 1, 2, 3, 4 };
45     System.out.println(ps.canPartition(num));
46     num = new int[] { 1, 1, 3, 4, 7 };
47     System.out.println(ps.canPartition(num));
48     num = new int[] { 2, 3, 4, 6 };
49     System.out.println(ps.canPartition(num));
50 }
51 }

```

Run

Save

Reset



Time and Space complexity

The above algorithm has the time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

Bottom-up Dynamic Programming

Let's try to populate our `dp[][]` array from the above solution by working in a bottom-up fashion. Essentially, we want to find if we can make all possible sums with every subset. **This means, `dp[i][s]` will be 'true' if we can make the sum 's' from the first 'i' numbers.**

So, for each number at index 'i' ($0 \leq i < \text{num.length}$) and sum 's' ($0 \leq s \leq S/2$), we have two options:

1. Exclude the number. In this case, we will see if we can get 's' from the subset excluding this number: `dp[i-1][s]`.
2. Include the number if its value is not more than 's'. In this case, we will see if we can find a subset to get the remaining sum: `dp[i-1][s-num[i]]`.

If either of the two above scenarios is true, we can find a subset of numbers with a sum equal to 's'.

Let's start with our base case of zero capacity:

num\sum	0	1	2	3	4	5
1	T					
{1, 2}	T					
{1,2,3}	T					
{1,2,3,4}	T					

'0' sum can always be found through an empty set

1 of 10

From the above visualization, we can clearly see that it is possible to partition the given set into two subsets with equal sums, as shown by bottom-right cell: `dp[3][5] => T`

Code

Here is the code for our bottom-up dynamic programming approach:

Java

Python3

C++

JS

```

1 class PartitionSet {
2
3     public boolean canPartition(int[] num) {
4         int n = num.length;
5         // find the total sum
6         int sum = 0;
7         for (int i = 0; i < n; i++)
8             sum += num[i];
9
10        // if 'sum' is a an odd number, we can't have two subsets with same total
11        if(sum % 2 != 0)
12            return false;
13
14        // we are trying to find a subset of given numbers that has a total sum of 'sum/2'.
15        sum /= 2;
16
17        boolean[][] dp = new boolean[n][sum + 1];
18
19        // populate the sum=0 columns, as we can always for '0' sum with an empty set
20        for(int i=0; i < n; i++)
21            dp[i][0] = true;
22
23        // with only one number, we can form a subset only when the required sum is equal to its value
24        for(int s=1; s <= sum ; s++) {
25            dp[0][s] = (num[0] == s ? true : false);
26        }
27
28        // process all subsets for all sums

```

```

28 // process all subsets for all sums
29 for(int i=1; i < n; i++) {
30     for(int s=1; s <= sum; s++) {
31         // if we can get the sum 's' without the number at index 'i'
32         if(dp[i-1][s]) {
33             dp[i][s] = dp[i-1][s];
34         } else if (s >= num[i]) { // else if we can find a subset to get the remaining sum
35             dp[i][s] = dp[i-1][s-num[i]];
36         }
37     }
38 }
39
40 // the bottom-right corner will have our answer.
41 return dp[n-1][sum];
42 }
43
44 public static void main(String[] args) {
45     PartitionSet ps = new PartitionSet();
46     int[] num = {1, 2, 3, 4};
47     System.out.println(ps.canPartition(num));
48     num = new int[]{1, 1, 3, 4, 7};
49     System.out.println(ps.canPartition(num));
50     num = new int[]{2, 3, 4, 6};
51     System.out.println(ps.canPartition(num));
52 }
53 }
54

```

Run

Save

Reset



Time and Space complexity

The above solution has time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

[< Back](#)
☒ Mark As Completed

[Next >](#)

0/1 Knapsack (medium)

Subset Sum (medium)